

Python pour le trading — Document 8/8

La performance

1. Pourquoi Python peut être lent

Python privilégie la simplicité d'écriture sur la vitesse brute. C'est un langage **interprété** et **dynamique** : il vérifie beaucoup de choses au moment de l'exécution (le type de chaque variable, par exemple), ce qui a un coût. Une boucle qui calcule sur des millions d'éléments en Python pur sera donc nettement plus lente que le même code en C.

```
# Lent : boucle Python pure sur un gros volume
total = 0
for prix in liste_de_millions_de_prix:
    total += prix * 1.01      # Python re-vérifie le type à chaque tour
```

À chaque tour de boucle, Python refait un travail invisible (vérifier les types, gérer les objets), ce qui s'accumule sur des millions d'itérations. La bonne nouvelle : on n'a presque jamais besoin de quitter Python pour aller plus vite. Les sections suivantes montrent comment, dans l'ordre où il faut les essayer.

***Pourquoi ça compte.** Comprendre POURQUOI Python est lent oriente vers les bonnes solutions. Plutôt que de réécrire en C, on s'appuie sur des bibliothèques optimisées, ce qui garde la simplicité de Python tout en gagnant la vitesse là où elle compte.*

2. La vectorisation : la première solution

La solution la plus efficace et la plus simple est la **vectorisation** avec NumPy, vue au document 5. Au lieu de boucler en Python, on délègue le calcul à NumPy, qui l'exécute en code C optimisé sur tout le tableau d'un coup.

```
import numpy as np

prix = np.array(liste_de_millions_de_prix)

# Au lieu de la boucle Python lente :
total = (prix * 1.01).sum()    # NumPy fait tout en C, des dizaines de fois plus vite
```

La même opération que la boucle de la section 1, mais `(prix * 1.01).sum()` est exécutée entièrement par NumPy en code compilé, sans repasser par l'interpréteur Python à chaque élément. Sur de gros volumes, le gain est typiquement d'un facteur 10 à 100. C'est pourquoi la règle d'or est : **toujours essayer la vectorisation en premier**. Elle résout la grande majorité des problèmes de lenteur en analyse de données.

***Pourquoi ça compte.** Pour calculer des indicateurs ou des rendements sur de gros historiques, la vectorisation suffit presque toujours. C'est le premier réflexe performance, simple et très efficace, avant d'envisager des outils plus lourds.*

3. Le multiprocessing : pour le calcul parallèle

Quand le calcul reste lourd **après** vectorisation, et qu'il est CPU-bound, on répartit le travail sur plusieurs cœurs avec le **multiprocessing** vu au document 4. Rappel : à cause du GIL, le threading n'aide pas pour le calcul pur ; il faut des processus séparés.

```
from multiprocessing import Pool

def analyser(symbole_data):
    return calcul_lourd(symbole_data)    # CPU-bound, déjà vectorisé au mieux

if __name__ == "__main__":
    with Pool(4) as pool:                # 4 cœurs en parallèle
        resultats = pool.map(analyser, [es, nq, cl, gc])
```

On a déjà vu ce motif au document 4 : `Pool(4)` lance quatre processus qui calculent en parallèle réel, chacun sur un cœur. C'est complémentaire de la vectorisation : on vectorise d'abord chaque calcul, puis on répartit les calculs indépendants sur plusieurs cœurs. L'ordre des solutions est important : vectoriser, puis paralléliser seulement si c'est encore trop lent.

Pourquoi ça compte. Pour des backtests ou des analyses sur de nombreux symboles, combiner vectorisation et multiprocessing exploite pleinement la machine. C'est ce qui réduit des heures de calcul à des minutes en recherche quantitative.

4. numba et Cython : accélérer le code restant

Parfois, un calcul ne se vectorise pas facilement (une logique séquentielle complexe, par exemple). Deux outils permettent alors de **compiler** du code Python pour le rapprocher de la vitesse du C. À connaître surtout de nom, sans forcément les maîtriser.

4.1 numba : la compilation à la volée

```
from numba import jit

@jit                                # compile la fonction en code machine
def calcul_sequentiel(prix):
    total = 0.0
    for p in prix:                  # cette boucle devient quasi aussi rapide qu'en C
        total += p * 1.01
    return total
```

numba est remarquablement simple : il suffit d'ajouter le décorateur `@jit` (on reconnaît les décorateurs du document 3) au-dessus d'une fonction, et numba la **compile** en code machine au premier appel (*just-in-time*). Une boucle Python lente peut ainsi devenir presque aussi rapide qu'en C, sans réécrire le code. C'est idéal quand la vectorisation est impossible mais qu'on veut garder du Python lisible.

4.2 Cython : du Python compilé

Cython est une autre approche : il traduit du code Python (légèrement annoté) en C, qu'on compile ensuite. Plus puissant mais plus complexe que numba, il est utilisé pour les portions les plus critiques en performance. À ce stade, l'essentiel est de **savoir que ces outils existent** et qu'on y recourt seulement après avoir épuisé la vectorisation et le profilage.

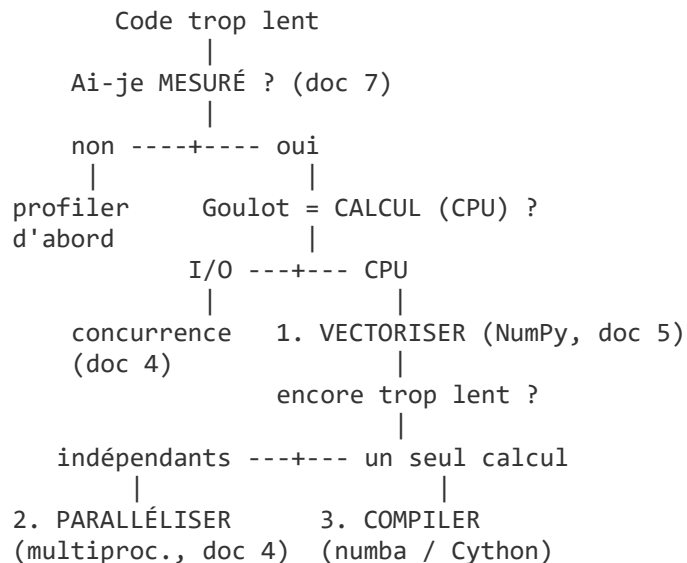
Pourquoi ça compte. Dans une firme HFT où la latence est un avantage concurrentiel, savoir qu'on peut compiler les portions critiques (numba, Cython) montre une conscience de la performance. Mais on n'y recourt qu'après avoir mesuré (document 7) et vectorisé : optimiser le bon endroit, au bon moment.

5. La démarche d'optimisation

La performance n'est pas une question d'astuces isolées, mais d'une **démarche ordonnée**. Voici l'ordre à suivre, qui résume les documents 7 et 8.

5.1 Arbre de décision : que faire quand c'est lent ?

Plutôt que de répéter les outils (vus aux documents 4 et 5), ce module sert surtout de **guide de décision**. Face à du code trop lent, on ne devine pas : on suit cet arbre.



On lit l'arbre de haut en bas. On **mesure** toujours en premier (document 7) : sans cela, on optimise à l'aveugle. La première bifurcation est décisive : un goulot I/O-bound (attente réseau, disque) relève de la concurrence du document 4 — threads ou asyncio — pas de la vitesse de calcul. Un goulot CPU-bound suit la chaîne **vectoriser → paralléliser → compiler**.

On **vectorise** d'abord (NumPy, document 5), ce qui suffit presque toujours. Si c'est encore trop lent, on **parallélise** les calculs indépendants sur plusieurs cœurs (multiprocessing, document 4).

On ne **compile** (numba / Cython) qu'en dernier recours, sur le peu de code séquentiel qui le justifie. La valeur de ce module n'est pas un outil de plus, mais l'**ordre** dans lequel les essayer.

1. MESURER d'abord (profilage, document 7)
-> ne jamais optimiser à l'aveugle ; trouver le vrai goulot.
2. VECTORISER (NumPy)
-> remplace les boucles Python ; résout la plupart des cas.
3. PARALLÉLISER (multiprocessing) si CPU-bound et encore trop lent
-> répartir sur plusieurs cœurs.
4. COMPILER (numba/Cython) pour les portions critiques restantes

-> en dernier recours, sur le peu de code qui le justifie.
Cet ordre est essentiel. On **mesure** d'abord pour savoir quoi optimiser (souvent, 90 % du temps est dans 10 % du code). On **vectorise**, qui suffit dans la majorité des cas. On ne **parallélise** puis ne **compile** que si la mesure le justifie. Optimiser prématurément complique le code pour un gain souvent nul : « la mesure d'abord » est la règle des développeurs expérimentés.

***Pourquoi ça compte.** Une démarche d'optimisation rigoureuse — mesurer, puis agir au bon endroit — est exactement l'état d'esprit attendu dans un environnement où la performance compte. Elle évite de complexifier le code inutilement et concentre l'effort là où il a un réel impact.*

Récapitulatif

1. **Pourquoi Python est lent** : langage interprété et dynamique ; les boucles pures sur de gros volumes coûtent cher.
2. **Vectorisation (NumPy)** : la première solution ; déléguer le calcul à du code C, facteur 10 à 100.
3. **Multiprocessing** : répartir le calcul CPU-bound sur plusieurs cœurs, après vectorisation.
4. **numba / Cython** : compiler le code restant qui ne se vectorise pas ; en dernier recours.
5. **La démarche** : mesurer, vectoriser, paralléliser, compiler — dans cet ordre.

Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous, en retrait.

Sur la lenteur de Python et la vectorisation

Q1. Pourquoi Python peut-il être lent sur de gros calculs ?

Réponse. Parce qu'il est interprété et dynamique : il vérifie beaucoup de choses à l'exécution (comme le type de chaque variable), ce qui a un coût qui s'accumule sur des millions d'itérations.

Q2. Quelle est la première solution à essayer pour accélérer un calcul ?

Réponse. La vectorisation avec NumPy : déléguer le calcul à du code C optimisé sur tout le tableau d'un coup, au lieu de boucler en Python. Gain typique d'un facteur 10 à 100.

Q3. A-t-on souvent besoin de quitter Python pour aller plus vite ?

Réponse. Non. Les bibliothèques optimisées (NumPy en tête) permettent de garder la simplicité de Python tout en gagnant la vitesse. On ne réécrit en C qu'en tout dernier recours, et rarement.

Sur le multiprocessing et la compilation

Q4. Quand recourir au multiprocessing pour la performance ?

Réponse. Quand le calcul est CPU-bound et reste trop lent après vectorisation. On répartit alors les calculs indépendants sur plusieurs cœurs (le threading n'aiderait pas, à cause du GIL).

Q5. Que fait numba, et avec quelle syntaxe ?

Réponse. numba compile une fonction Python en code machine au premier appel (just-in-time), ce qui accélère fortement les boucles. On l'active simplement avec le décorateur @jit au-dessus de la fonction.

Q6. Quand recourir à numba ou Cython plutôt qu'à la vectorisation ?

Réponse. Quand le calcul ne se vectorise pas facilement (logique séquentielle complexe). Ce sont des outils de dernier recours, pour les portions critiques restantes, après mesure.

Sur la démarche d'optimisation

Q7. Quel est le bon ordre des étapes d'optimisation ?

Réponse. Mesurer (profilage), puis vectoriser (NumPy), puis paralléliser (multiprocessing) si nécessaire, puis compiler (numba/Cython) en dernier recours.

Q8. Pourquoi mesurer avant d'optimiser ?

Réponse. Pour ne pas optimiser à l'aveugle. Souvent, 90 % du temps est passé dans 10 % du code ; le profilage révèle ce goulot, pour concentrer l'effort là où il a un réel impact.

Q9. Pourquoi l'optimisation prématurée est-elle un défaut ?

Réponse. Parce qu'elle complique le code pour un gain souvent nul, en optimisant des parties qui ne sont pas le vrai goulot. Mieux vaut un code simple, puis optimisé au bon endroit après mesure.

Fin de la série. Les huit documents couvrent l'ensemble des fondations Python attendues pour un poste de développeur en opérations de trading : fondamentaux, POO, concepts intermédiaires, concurrence, données, interactions, qualité et performance.